

3.2. Estructura básica de una aplicación Flutter



DESARROLLO DE APLICACIONES WEB MULTIPLATAFORMA NIVEL 3.

UNIDAD 3:

INTRODUCCION A FLUTTER

Contenido

Introducción.....	3
Árbol de widgets	3
Importancia de la estructura básica	4
Arquitectura básica de una aplicación Flutter	5
Representación visual	5
Detalle de los componentes	6
AppWidget	6
MaterialApp: Widget base para aplicaciones Flutter	7
Scaffold: Estructura principal de la interfaz de usuario	7
Home Widget: Contenido principal de la pantalla	7
StatelessWidget vs. StatefulWidget: Propiedades y rendimiento	7
Rutas y navegación entre pantallas	8
Entorno de desarrollo local	9
Creación de proyectos Flutter	9
Estructura de un proyecto Flutter	10
Archivos y directorios importantes	11
Recursos adicionales.....	12

Introducción

En el mundo de Flutter, los widgets son los bloques de construcción fundamentales para crear interfaces de usuario. Cada widget representa un elemento visual o interactivo en la pantalla, como un botón, un texto, una imagen o un contenedor.

Los widgets se pueden combinar y anidar entre sí para crear interfaces complejas y jerárquicas.

Árbol de widgets

La estructura de una aplicación Flutter se representa mediante un árbol de widgets. Cada widget es un nodo en este árbol, y cada nodo puede tener hijos, que son otros widgets.

El árbol de widgets define la jerarquía y la relación entre los elementos de la interfaz de usuario.

Ejemplo de Árbol de Widgets:

AppWidget (Raíz)

```

|
+-- Scaffold
|  |
|  +-- AppBar
|  |  |
|  |  +-- Text("Mi Aplicación")
|  |
|  +-- Body
|     |
|     +-- Center
|        |
|        +-- Text("¡Hola Mundo!")
  
```

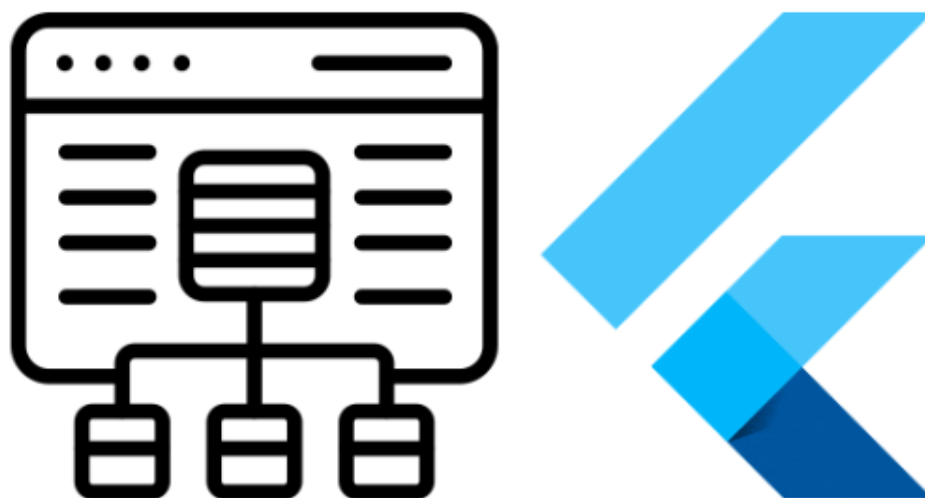
Importancia de la estructura básica

Comprender la estructura básica de una aplicación Flutter es crucial por varias razones:

- Organización y Mantenibilidad: Una estructura organizada facilita la lectura, comprensión y modificación del código, especialmente en proyectos grandes y complejos.
- Rendimiento y Escalabilidad: Una estructura bien definida permite optimizar el rendimiento de la aplicación y facilitar su crecimiento en el futuro.
- Experiencia de Usuario: La estructura básica influye en la disposición y el comportamiento de los elementos en la pantalla, impactando directamente en la experiencia del usuario.

En resumen, dominar la estructura básica de una aplicación Flutter es esencial para crear interfaces de usuario robustas, eficientes y amigables con el usuario.

Arquitectura básica de una aplicación Flutter



La arquitectura básica se refiere a la organización interna de una aplicación Flutter, definiendo la jerarquía de widgets y la relación entre ellos.

Es un concepto conceptual, independiente de la implementación física de los archivos del proyecto que son la estructura de carpetas del proyecto.

Se centra en los componentes funcionales de la aplicación y cómo interactúan entre sí.

Abordaremos este apartado incluyendo primero una representación visual para tener la visión de conjunto y posteriormente haremos una aproximación de detalle para cada uno de esos elementos.

Representación visual

La arquitectura básica sería la siguiente:

AppWidget (Raíz)

└─

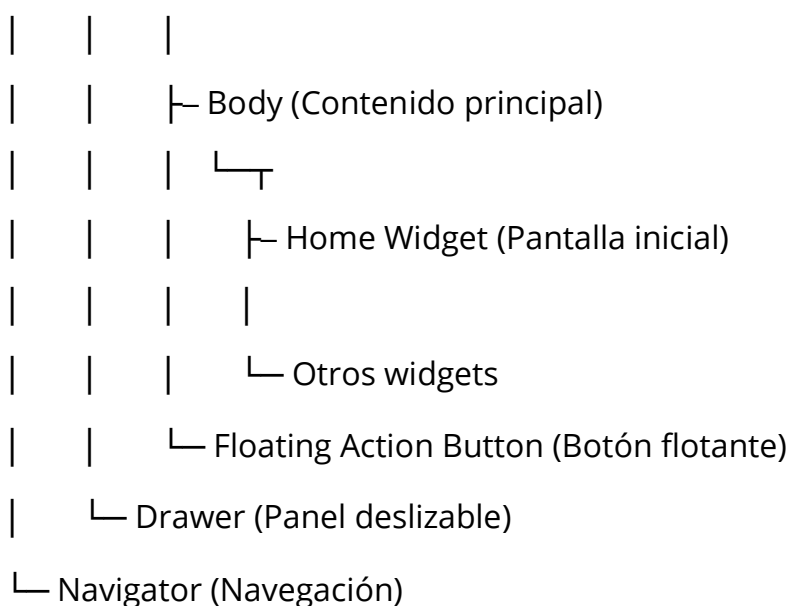
└─ MaterialApp (Base)

| └─

| └─ Scaffold (Estructura)

| | └─

| | └─ AppBar (Barra de herramientas)



Explicación:

- El widget raíz AppWidget crea la aplicación y contiene todo el árbol de widgets.
- MaterialApp proporciona características básicas como el tema y la navegación.
- Scaffold define la estructura principal de la interfaz de usuario con AppBar, Body, etc.
- Home Widget es un widget común para el contenido inicial de Body.
- Otros widgets se pueden anidar dentro de Body para crear la interfaz.
- Floating Action Button y Drawer son elementos opcionales dentro de Scaffold.
- Navigator es accesible desde cualquier parte de la aplicación para gestionar la navegación.

Detalle de los componentes

Veamos ahora los aspectos más relevantes de cada uno de esos elementos:

AppWidget

Toda aplicación Flutter comienza con un widget raíz llamado AppWidget. Este widget es el punto de entrada de la aplicación y define la estructura general de la interfaz de usuario. Es responsable de crear y administrar el árbol de widgets de la aplicación.

MaterialApp: Widget base para aplicaciones Flutter

En la mayoría de las aplicaciones Flutter, se utiliza el widget MaterialApp como widget raíz. Este widget proporciona características esenciales para aplicaciones Flutter, como:

- Tema de Material Design: Define la apariencia general de la aplicación, incluyendo colores, tipografías y estilos.
- Rutas y Navegación: Proporciona un sistema para gestionar la navegación entre diferentes pantallas de la aplicación.
- Gestión del Estado: Ofrece herramientas para manejar el estado de la aplicación y actualizar la interfaz de usuario en consecuencia.

Scaffold: Estructura principal de la interfaz de usuario

El widget Scaffold define la estructura principal de la interfaz de usuario de una aplicación Flutter. Proporciona elementos comunes como:

- AppBar: Barra de herramientas en la parte superior de la pantalla, que puede incluir un título, un logotipo o acciones.
- Body: Área principal de la pantalla que contiene el contenido principal de la aplicación.
- Floating Action Button: Botón flotante que se puede utilizar para realizar acciones importantes.
- Drawer: Panel deslizable que se puede utilizar para mostrar opciones de navegación o información adicional.

Home Widget: Contenido principal de la pantalla

El widget Home se utiliza para definir el contenido principal de la pantalla inicial de la aplicación. Este widget se configura dentro del widget MaterialApp y suele ser el primer widget que se muestra al usuario.

StatelessWidget vs. StatefulWidget: Propiedades y rendimiento

Los widgets en Flutter se clasifican en dos categorías principales:

- StatelessWidget: Son widgets con propiedades inmutables, lo que significa que no cambian después de crearse. Son más eficientes en términos de rendimiento y se utilizan para representar elementos estáticos de la interfaz de usuario.
- StatefulWidget: Son widgets con propiedades mutables, lo que significa que pueden cambiar con el tiempo. Se utilizan para

representar elementos dinámicos de la interfaz de usuario que requieren actualizaciones frecuentes.

La elección entre StatelessWidget y StatefulWidget depende de la naturaleza del widget y de cómo se comporte en la aplicación.

Rutas y navegación entre pantallas

La navegación entre diferentes pantallas es una funcionalidad esencial en cualquier aplicación móvil. Flutter proporciona un sistema de navegación basado en el widget Navigator. Este widget permite:

- Definir rutas: Cada pantalla de la aplicación se asocia con una ruta única.
- Navegar entre pantallas: Se pueden utilizar diferentes métodos para navegar entre pantallas, como push, pushReplacement, pop, etc.
- Pasar argumentos: Se pueden pasar datos entre diferentes pantallas utilizando argumentos de navegación.

Entorno de desarrollo local



El entorno de desarrollo local en Flutter se refiere al conjunto de herramientas y configuraciones que se instalan en tu ordenador para crear, probar y ejecutar aplicaciones Flutter. Este entorno te permite trabajar en tus proyectos de forma local, sin necesidad de depender de servicios en la nube o herramientas externas.

El entorno de desarrollo local juega un papel fundamental en el proceso de desarrollo de aplicaciones Flutter, ya que te permite:

- Escribir y editar código Dart: Utilizar un editor de código como Android Studio o VSCode para escribir y modificar el código fuente de tu aplicación Flutter.
- Diseñar la interfaz de usuario: Utilizar widgets y herramientas de diseño para crear la interfaz de usuario de tu aplicación.
- Probar y depurar tu código: Ejecutar tu aplicación en diferentes plataformas (simuladores, emuladores o dispositivos físicos) para identificar y corregir errores.
- Generar el código nativo: Compilar tu código Dart en código nativo para que pueda ejecutarse en dispositivos Android, iOS, web, escritorio (Windows, macOS, Linux) y Fuchsia.

Creación de proyectos Flutter

Para crear un proyecto Flutter, puedes seguir estos pasos:

1. Abre tu IDE de desarrollo (Android Studio o VSCode).
2. Ve a File > New > Project.

3. Selecciona Flutter Application en la lista de plantillas.
4. Introduce el nombre de tu proyecto y haz clic en Finish.

Estructura de un proyecto Flutter

La estructura de un proyecto refiere a la organización física de los archivos del proyecto Flutter, definiendo cómo se almacenan y estructuran los distintos componentes del código.

Un proyecto Flutter típico tiene la siguiente estructura:

project_name/

```
├── android/
|   ├── app/
|   |   ├── src/
|   |   |   └── main.dart
|   |   └── AndroidManifest.xml
|   └── build.gradle
├── gradle.properties
├── ios/
|   ├── Runner.xcworkspace
|   ├── Runner.xcodeproj
|   └── Podfile
├── lib/
|   ├── main.dart
|   └── widgets/
|       └── your_widgets.dart
├── pubspec.yaml
├── README.md
└── test/
```

└─ widget_test.dart

Explicación:

- android: Contiene los archivos y recursos específicos para la plataforma Android.
- ios: Contiene los archivos y recursos específicos para la plataforma iOS.
- lib: Contiene el código Dart principal de tu aplicación, incluyendo el archivo main.dart y los widgets personalizados.
- pubspec.yaml: Define las dependencias de tu proyecto, como bibliotecas y paquetes externos.
- README.md: Contiene información sobre el proyecto, como instrucciones de instalación y uso.
- test: Contiene las pruebas unitarias de tu código.

Archivos y directorios importantes

Algunos de los archivos y directorios más importantes en un proyecto Flutter son:

- main.dart: El punto de entrada de tu aplicación, donde se define la clase runApp() que inicia la aplicación.
- widgets/: Contiene los widgets personalizados que utilizas en tu aplicación.
- pubspec.yaml: Define las dependencias de tu proyecto, como bibliotecas y paquetes externos.
- AndroidManifest.xml: Define la configuración de la aplicación para la plataforma Android.
- Info.plist: Define la configuración de la aplicación para la plataforma iOS.

Recursos adicionales

Documentación oficial de Flutter: <https://docs.flutter.dev/>

Tutoriales de Flutter: <https://docs.flutter.dev/get-started/codelab>

Comunidad de Flutter: <https://flutter.dev/community>

Foros de Flutter: <https://meta.discourse.org/t/dart-flutter-code-block-support/250122>

Estructura de un proyecto Flutter:
<https://medium.com/googledeveloperseurope/clean-architecture-en-flutter-ee028a6379a5>

¡Enhorabuena por completar esta unidad didáctica sobre la estructura básica de una aplicación Flutter!

En este módulo, hemos explorado los conceptos fundamentales que forman la base de cualquier aplicación Flutter. Dominar estos conceptos es esencial para construir aplicaciones Flutter robustas, escalables y fáciles de usar.

¡No dudes en seguir aprendiendo y explorando el mundo de Flutter!

¡Hasta la próxima!